

Performance Analysis

- **Amdahl's Law:** fixed problem size $\Rightarrow$ speedup limited by the serial fraction (strong scaling, pessimistic bound).
- **Gustafson's Law:** scaled problem size $\Rightarrow$ speedup grows with processors as parallel work increases (weak scaling, optimistic view).
- **Karp-Flatt metric:** empirical measure $\Rightarrow$ extracts effective serial fraction from observed speedup (diagnoses overhead and scalability).

Isoefficiency

- When we increase the number of processes for a fixed problem size, the efficiency typically decreases because communication overhead and synchronization costs grow.
- If we increase both the problem size and the number of processors, we can often maintain a constant efficiency.
- Isoefficiency answers the question: “How must the problem size grow to maintain constant efficiency?”

Derivation

- $W = T_s$ (serial time or total work)
- Total overhead: $T_o = p T_p = W$
- Parallel runtime: $T_p = (W + T_o) / p$
- Efficiency: $E = W / (p T_p) = W / (W + T_o)$
- Solving for W :

(Homework)

$$W = K T_o$$

where $K = E / (1 - E)$

Example

- Work: n additions (W)
- Communication: Uses a binary tree reduction. Each of the $\log p$ stages involves communication.
- Overhead T_o : Each of the p processes spends $O(\log p)$ time on communication $\Rightarrow T_o = O(p \log p)$.
- To keep constant efficiency, force the work to be proportional to the overhead, i.e.,

$$W = K (p \log p) = n$$

- Because n must grow at the rate of $p \log p$ to keep the processes busy, the isoefficiency function is $O(p \log p)$.

Interpretation

- $O(p)$, highly scalable $\Rightarrow$ Only need to increase the data proportionally to the processes.
- $O(p \log p)$, scalable $\Rightarrow$ Slight increase in data per process is needed.
- $O(p^2)$, poorly scalable $\Rightarrow$ Need to quadrupled the data every time you double the processes.
- Exponential, non-scalable $\Rightarrow$ The algorithm will fail to keep up on large clusters.

CPSC 375 High-Performance Computing

Lecture 28

Parallel Sorting Algorithms II

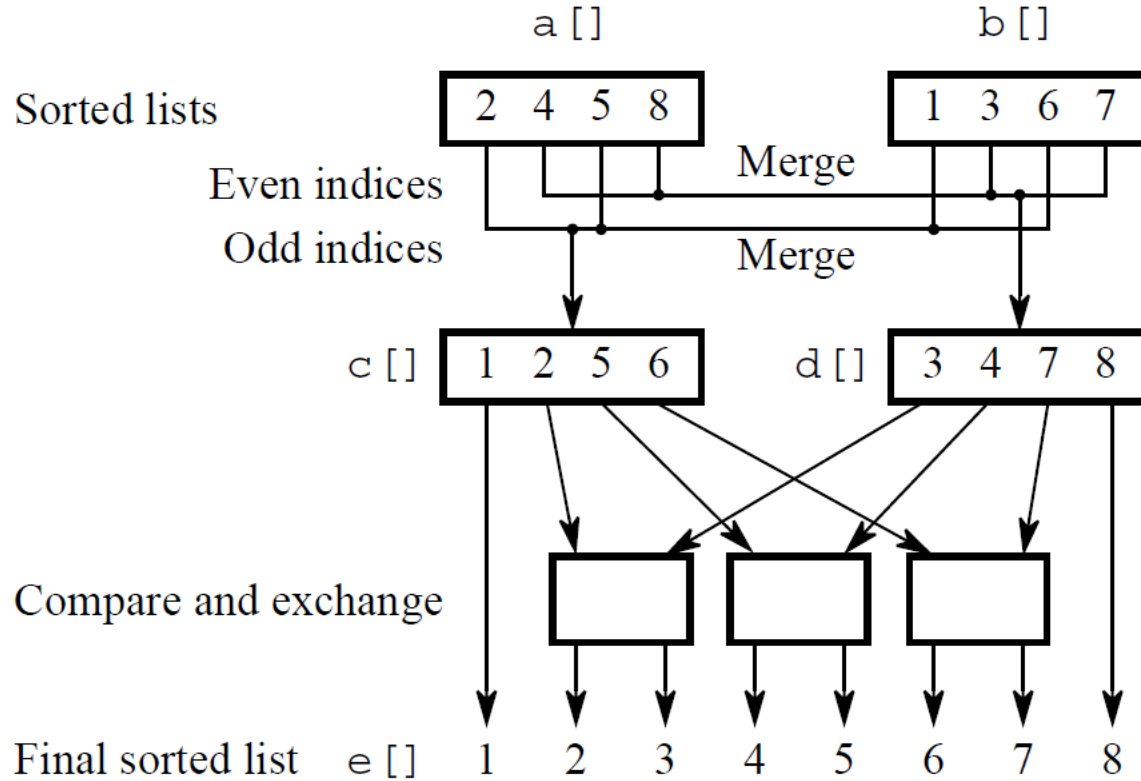
Batcher's Parallel Sorting Algorithms

- Odd-even Mergesort
- Bitonic Mergesort
- Originally derived in terms of *switching networks*.
- Both are well-balanced and have parallel time complexity of $O(\log^2 n)$ with n processors.

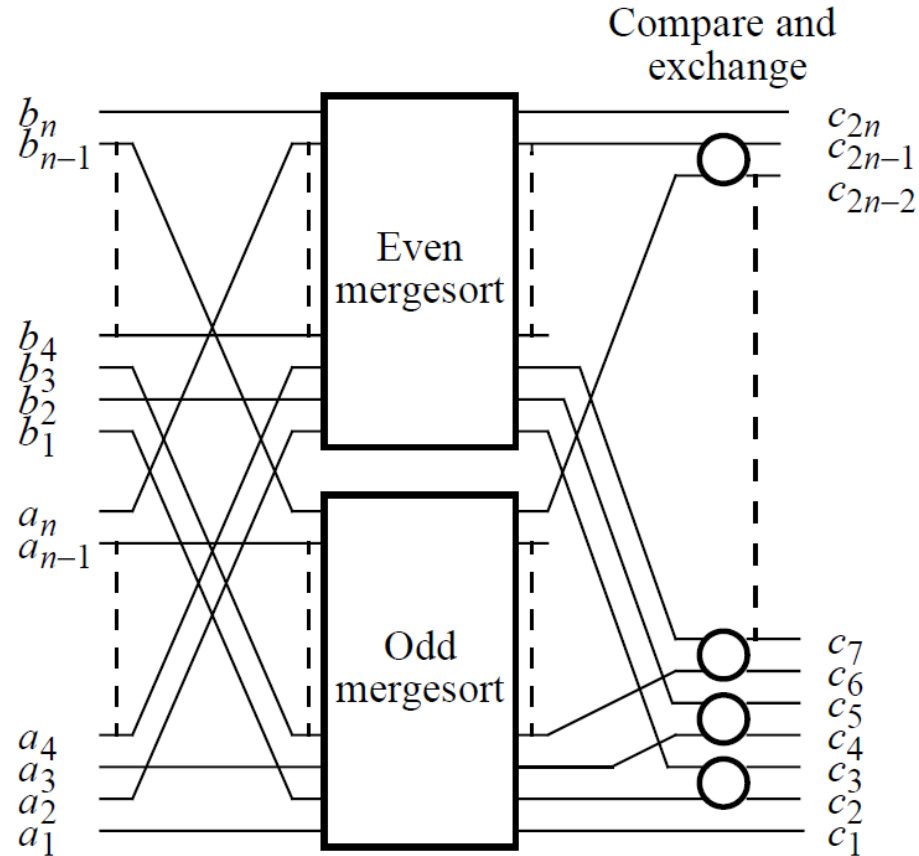
Odd-Even Mergesort

- Start with odd-even merge algorithm which will merge two sorted lists into one sorted list.
- Given two sorted lists $a_1, a_2, a_3, \dots, a_n$ and $b_1, b_2, b_3, \dots, b_n$ (where n is a power of 2)
- Elements with odd indices of each sequence, $a_1, a_3, \dots, a_{n-1}$ and $b_1, b_3, \dots, b_{n-1}$ are merged into one sorted list, $c_1, c_2, c_3, \dots, c_n$.
- Elements with even indices of each sequence, $a_2, a_4, \dots, a_n$, and $b_2, b_4, \dots, b_n$ are merged into one sorted list, $d_1, d_2, d_3, \dots, d_n$.
- The final sorted list, $e_1, e_2, e_3, \dots, e_n$ is obtained by:
$$e_{2i} = \min\{c_{i+1}, d_i\} \quad e_{2i+1} = \max\{c_{i+1}, d_i\}$$

Odd-Even Mergesort, Example



Odd-Even Mergesort



Odd-Even Mergesort

- Essentially the odd and even index lists are interleaved, and pairs of odd/even elements are interchanged to move the larger toward one end, if necessary.
- First number is given by $e_1 = c_1$ (since this will be the smallest of the first elements of each list, a_1 or b_1)
- Last number by $e_{2n} = d_n$ (since this will be the largest of the last elements of each list, a_n or b_n).
- Pairs of processors perform the compare-and-exchange operations.
- Can be implemented with hardware units that perform the compare-and-exchange operations.

Bitonic Mergesort

Bitonic Sequence

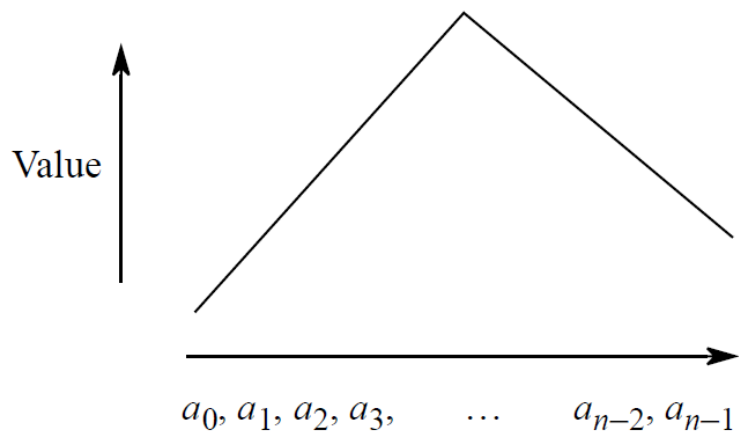
- A monotonic increasing sequence is a sequence of increasing numbers.
- A *bitonic sequence* has two sequences, one increasing and one decreasing. e.g.

$$a_0 < a_1 < a_2, a_3, \dots, a_{i-1} < a_i > a_{i+1}, \dots, a_{n-2} > a_{n-1}$$

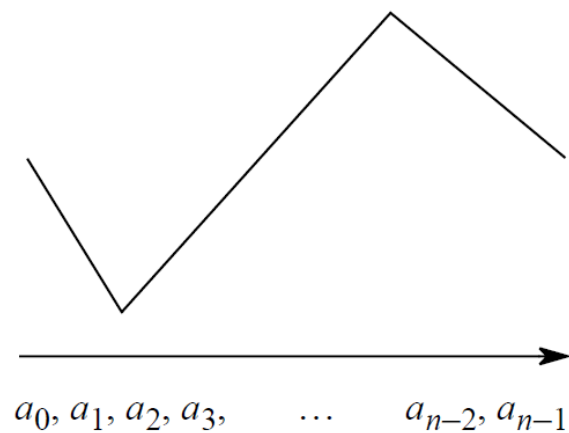
for some value of i ($0 \leq i < n$).

- A sequence is also bitonic if the preceding can be achieved by shifting the numbers cyclically (left or right).

Bitonic Sequences



Single maximum

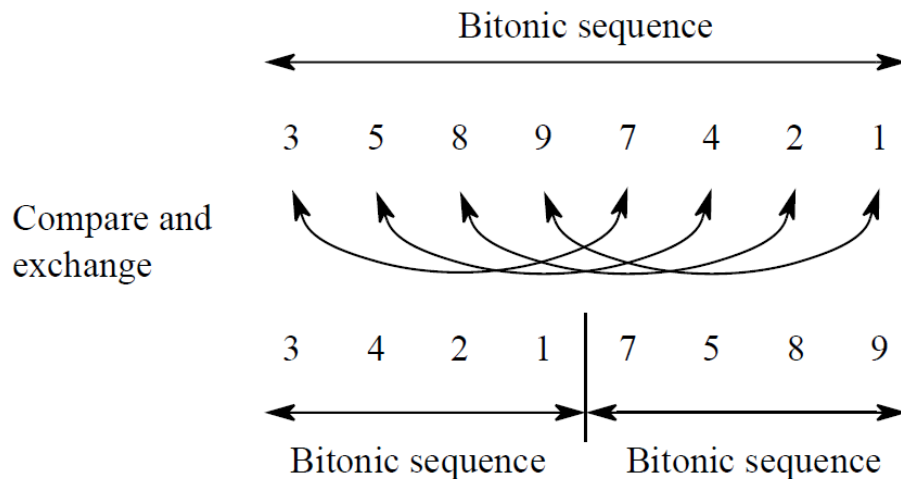


Single maximum and single minimum

Bitonic Sequences

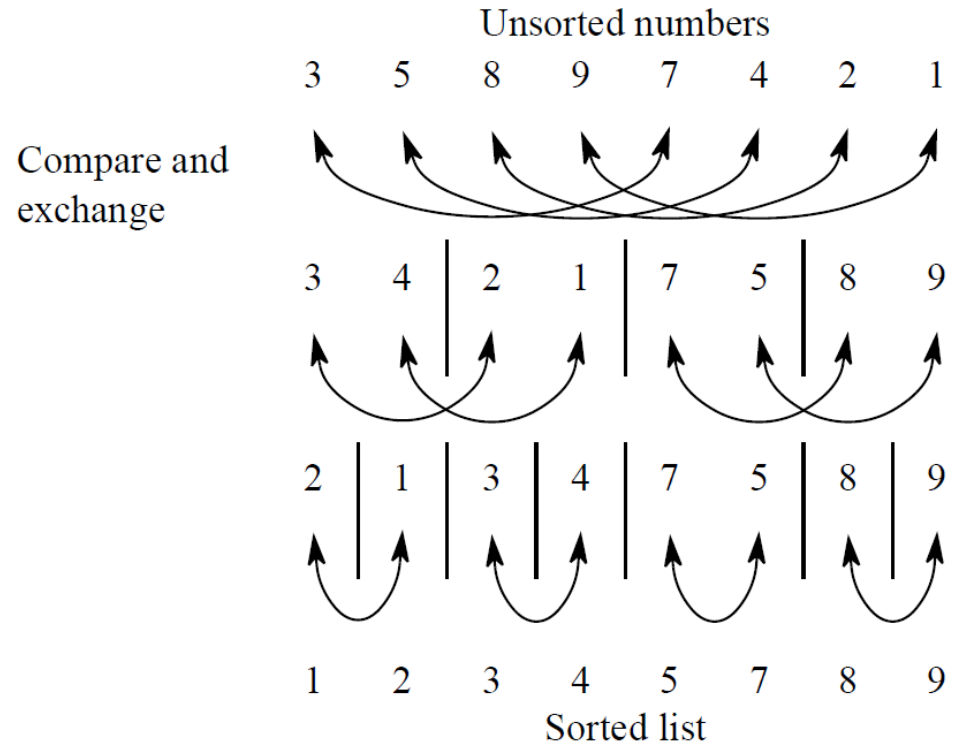
If we perform a compare-and-exchange operation on a_i with $a_{i+n/2}$

For all i , where there are n numbers in the sequence, get **two** bitonic sequences, where the numbers in one sequence are all less than the numbers in the other sequence.



Sorting Bitonic Sequences

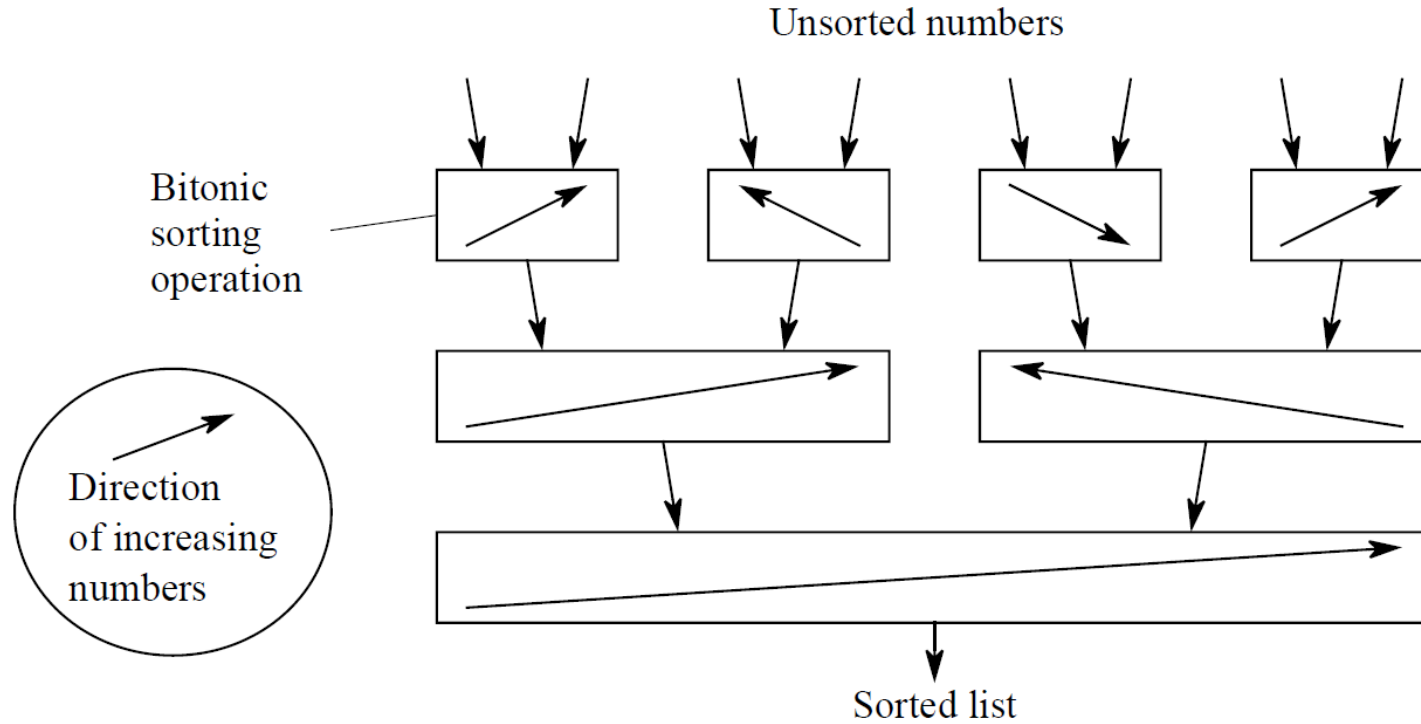
- Compare-and-exchange moves smaller numbers of each pair to left and larger numbers of pair to right.
- Given a bitonic sequence, recursively performing operations will sort the list.



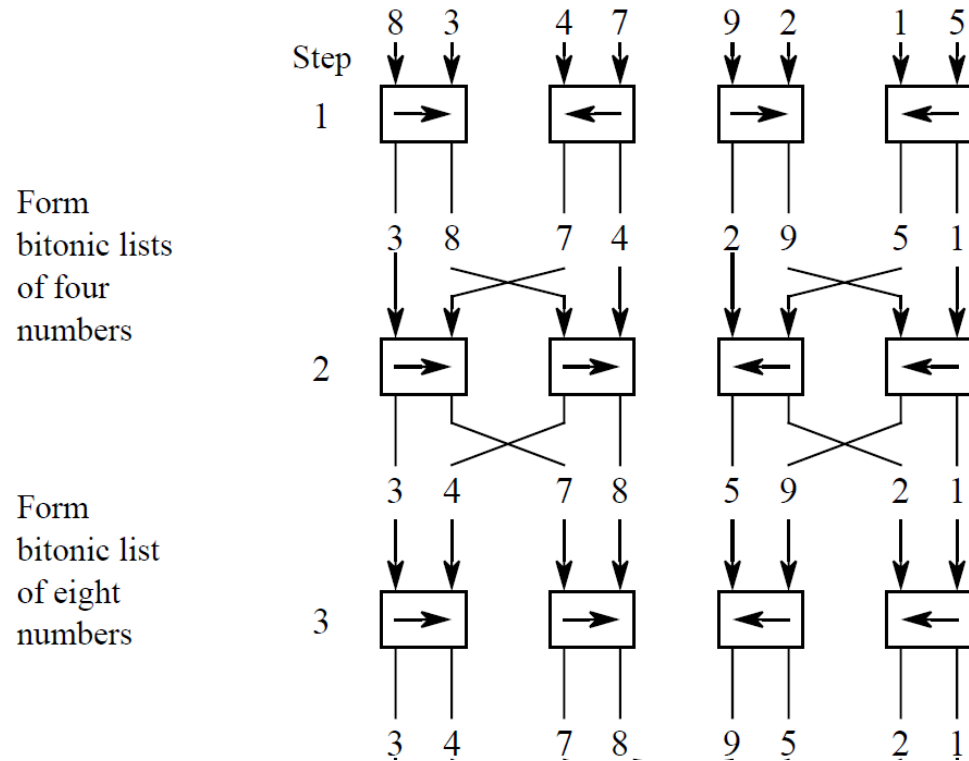
Bitonic Sorting

1. To sort an unordered sequence, sequences are merged into larger bitonic sequences, starting with pairs of adjacent numbers.
2. By a compare-and-exchange operation, pairs of adjacent numbers are formed into increasing sequences and decreasing sequences, pairs of which form a bitonic sequence of twice the size of each of the original sequences.
3. By repeating this process, bitonic sequences of larger lengths are obtained.
4. In the final step, a single bitonic sequence is sorted into a single increasing sequence.

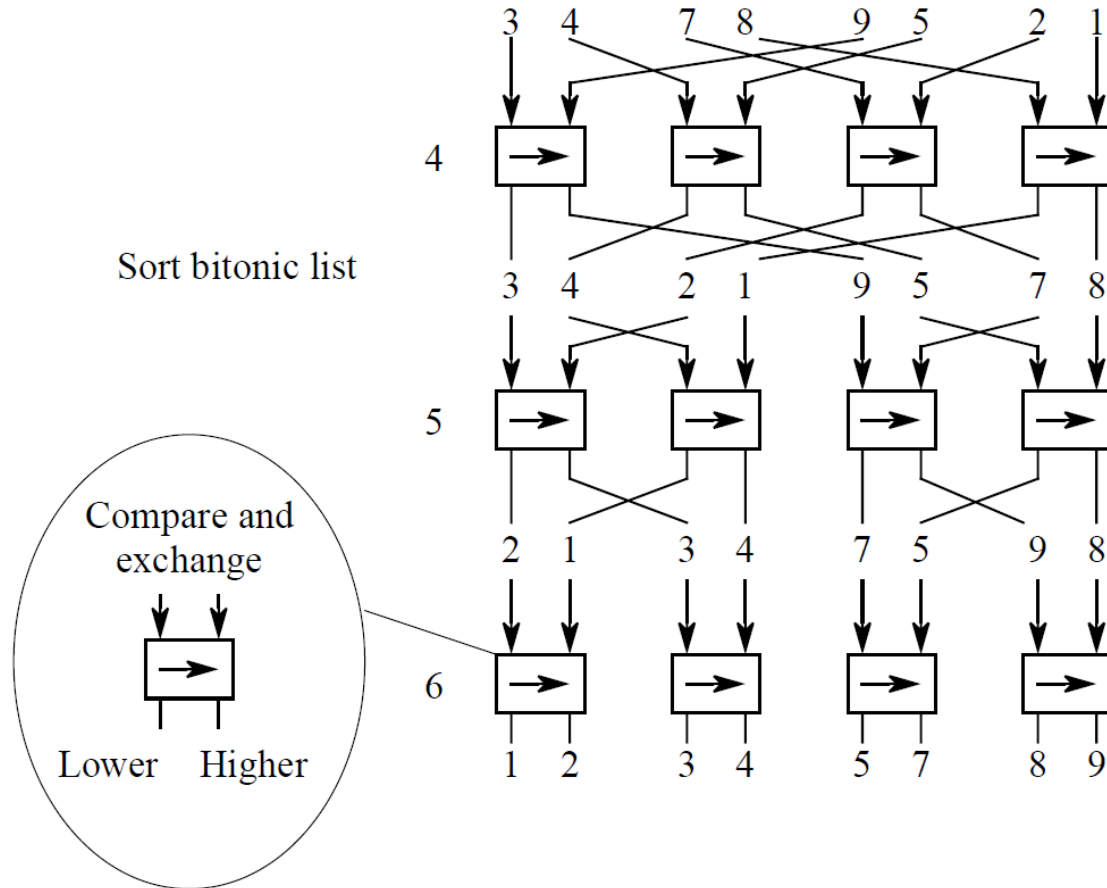
Bitonic Sorting



Bitonic Sorting, Example



Bitonic Sorting, Example



Bitonic Sorting, Phases

The six steps (for eight numbers) are divided into three phases:

Phase 1. Convert pairs of numbers into increasing/ decreasing sequences and hence into 4-bit bitonic sequences.

Phase 2. Split each 4-bit bitonic sequence into two 2-bit bitonic sequences, higher sequences at center.
Sort each 4-bit bitonic sequence increasing/ decreasing sequences and merge into 8-bit bitonic sequence.

Phase 3 Sort 8-bit bitonic sequence.

Bitonic Sorting, Analysis

In general, with $n = 2^k$, there are k phases, each of 1, 2, 3, ..., k steps. Hence the total number of steps is given by

$$\text{Steps} = \sum_{i=1}^k i = \frac{k(k+1)}{2} = \frac{\log n (\log n + 1)}{2} = O(\log^2 n)$$

